

Architecture Decision Record

Decision

One architectural decision that we made as a group was to utilize React and its component-based architecture to build the web interface of our application. We also decided to use Chakra UI (a component library) and Vite (a build tool) in conjunction with React.

Context

We knew that we wanted to create a web interface for our application, and therefore knew that we would need some sort of framework or library that would allow us to achieve this.

Options Considered

React seemed like a strong choice from the beginning, but we also considered other options including alternative frameworks/libraries (Vue, Angular, etc.), templating engines (Handlebars, Jinja, etc), and Vanilla JS (likely used alongside a templating engine).

Rationale

A few factors led to our decision to use React:

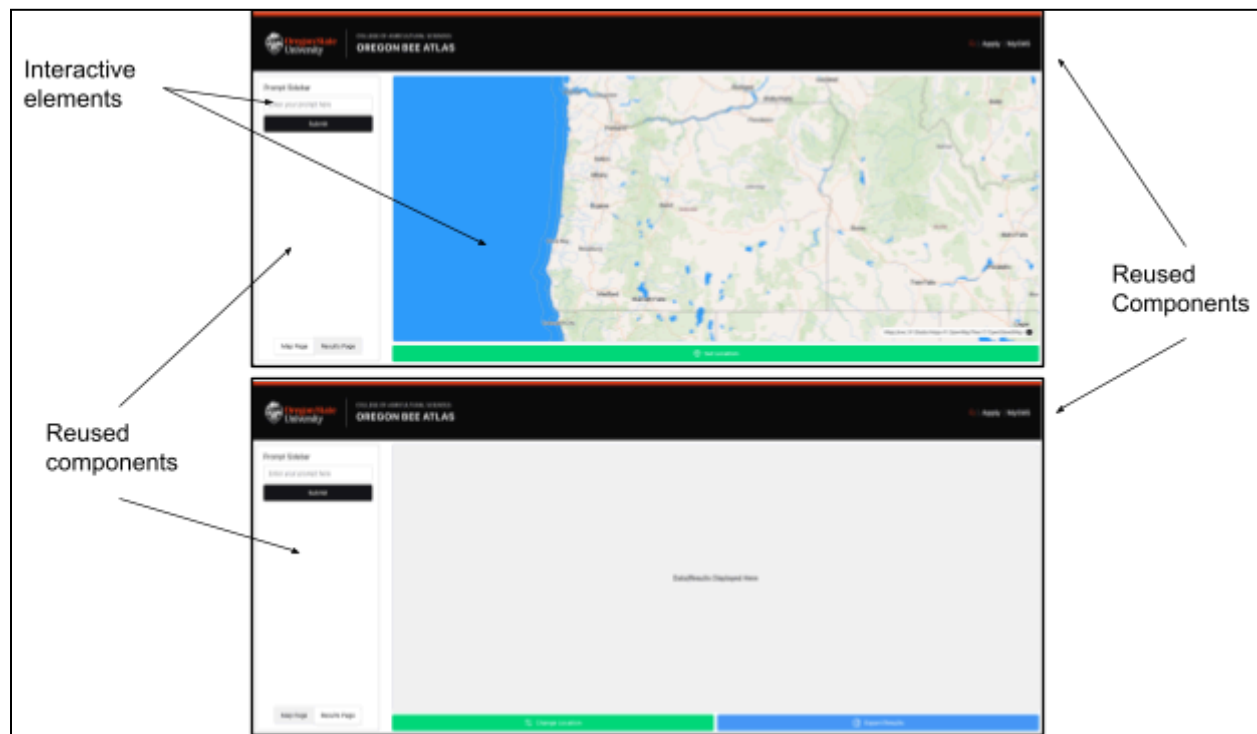
- Members' familiarity/prior use
- Component-based architecture - we knew we would likely have different views/pages in our application, but some common components such as a banner and a sidebar. By creating reusable components in React, we can efficiently and systematically structure our interface.
- Nature of our application - we knew we would likely be creating an application with some degree of interactivity, as opposed to static content that could be handled by templating/Vanilla JS/HTML.

Consequences

While React and its component-based architecture offer many benefits, there are some potential drawbacks of this decision:

- More tools and complexity are involved - React, along with the Chakra UI component library and Vite build tool, offer a lot of development benefits, but also require more setup and introduce more complexity.
- Learning curve - although members of our team have solid experience with React, it can still be challenging to debug and implement in some scenarios.

Evidence



The figure above shows some of the interface design choices (across 2 different application “views” or pages) that React has enabled us to make. While it may be possible to replicate this using a different framework or approach, React has enabled us to easily create a modular component-based interface for our application and will likely offer performance benefits once interactivity is introduced, as shown above.